

PABLO LOPEZ FERNANDEZ

TFG_Memoria_Final_Definitiva.pdf

-  Turnitin Memoria Final
-  TFG ETSIINF (Moodle PP)
-  Universidad Politecnica de Madrid

Detalles del documento

Identificador de la entrega

trn:oid:::1:3586875416

Fecha de entrega

4 jun 2026, 7:30 p.m. GMT+2

Fecha de descarga

4 jun 2026, 7:37 p.m. GMT+2

Nombre del archivo

18178_PABLO_LOPEZ_FERNANDEZ_TFG_Memoria_Final_Definitiva_530462_1973335799.pdf

Tamaño del archivo

1.1 MB

43 páginas

10.704 palabras

65.949 caracteres

7% Similitud general

El total combinado de todas las coincidencias, incluidas las fuentes superpuestas, para ca...

Fuentes principales

- 7%  Fuentes de Internet
 - 2%  Publicaciones
 - 0%  Trabajos entregados (trabajos del estudiante)
-

Fuentes principales

7%	Fuentes de Internet
2%	Publicaciones
0%	Trabajos entregados (trabajos del estudiante)

Fuentes principales

Las fuentes con el mayor número de coincidencias dentro de la entrega. Las fuentes superpuestas no se mostrarán.

1	Internet	oa.upm.es	2%
2	Internet	www.rss-scout.de	<1%
3	Internet	www.scribd.com	<1%
4	Internet	documentop.com	<1%
5	Internet	www.coursehero.com	<1%
6	Internet	d197for5662m48.cloudfront.net	<1%
7	Publicación	Simone Cirani, Marco Picone, Pietro Gonizzi, Luca Veltri, Gianluigi Ferrari. "IoT-OA...	<1%
8	Internet	elibrary.math.rs	<1%
9	Internet	http://146.83.6.6/bdmc/literatura/memoria_jlcerda.pdf	<1%
10	Internet	issuu.com	<1%
11	Publicación	Venkata Gunnu, Shubham Shah, Anvesh Minukuri, Jayanth Gopu. "Practical Soluti...	<1%

12	Internet	developers.lseg.com	<1%
13	Internet	kainhofer.com	<1%
14	Internet	kualitaconsultores.com	<1%
15	Internet	kth.diva-portal.org	<1%
16	Internet	skemman.is	<1%
17	Internet	www.nursinghero.com	<1%
18	Internet	docs.appdynamics.com	<1%
19	Internet	ijetrm.com	<1%
20	Internet	www.bsc.es	<1%
21	Internet	www.ijprems.com	<1%
22	Internet	etd.repository.ugm.ac.id	<1%
23	Internet	www.slideshare.net	<1%
24	Publicación	Dmytro Kozhevin. "Building Serverless Robotics with AWS, AI, and ROS 2", Springe...	<1%
25	Internet	docta.ucm.es	<1%

26	Internet	en.sipiapa.org	<1%
27	Internet	www.bbc.com	<1%
28	Internet	1library.co	<1%
29	Internet	core.ac.uk	<1%
30	Internet	datatracker.ietf.org	<1%
31	Internet	hdl.handle.net	<1%
32	Internet	idus.us.es	<1%
33	Internet	profesores.elo.utfsm.cl	<1%
34	Internet	www.corpoboyaca.gov.co	<1%
35	Publicación	Nick Alteen, Jennifer Fisher, Casey Gerena, Wes Gruver, Asim Jalis, Heiwad Osman...	<1%



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Trabajo Fin de Grado

Integración de Calendarios de Diferentes Servicios con Arquitectura sin servidor (*Serverless*) y Despliegue Automatizado

Autor: Pablo López Fernández
Tutor: Miguel Jiménez Gañan

Madrid, 06/2026

1

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: Integración de Calendarios de Diferentes Servicios con Arquitectura Serverless y Despliegue Automatizado

Mes Año 06/2026

Autor: Pablo López Fernández

Tutor:

Miguel Jiménez Gañan

Departamento Universitario Lenguajes y Sist. Informáticos ETSI Informáticos

Universidad Politécnica de Madrid

1

Resumen

La gestión del tiempo mediante servicios de calendario en la nube se ha convertido en una práctica habitual tanto en el ámbito personal como profesional. Sin embargo, cuando un usuario trabaja con múltiples cuentas o proveedores distintos —Google Calendar, Apple iCloud o cualquier servidor CalDAV—, la visualización unificada de todos sus eventos resulta especialmente problemática en entornos de escritorio basados en navegador web, donde cada pestaña queda ligada a una única sesión de usuario.

El presente Trabajo de Fin de Grado aborda este problema mediante el diseño e implementación de CalendarFusion, una herramienta web que permite integrar calendarios procedentes de diferentes cuentas y servicios, aplicar filtros y transformaciones sobre los eventos —como prefijos y sufijos en el título, o exclusión por palabras clave—, y exponer el resultado en formato ICS/WebCal para que pueda ser suscrito desde Google Calendar, Microsoft Outlook o Apple Calendar.

La arquitectura de la solución es completamente *serverless* sobre AWS: el *backend* es un microservicio Java/Spring Boot empaquetado como función AWS Lambda e invocado a través de API Gateway; la persistencia de configuraciones se realiza en DynamoDB; el caché de ficheros ICS generados se almacena en S3; y el refresco periódico de las fusiones se activa mediante reglas de EventBridge. La interfaz de usuario es una SPA desarrollada con Angular, desplegada en S3 y distribuida globalmente mediante CloudFront con HTTPS. Toda la infraestructura se define de forma declarativa con Terraform, lo que permite su reproducción completa en un único comando.

Se ha implementado soporte para la autenticación OAuth 2.0 con Google Calendar API y para cualquier servidor CalDAV compatible, incluyendo Apple iCloud. La herramienta incorpora lógica de prevención de circularidades para evitar la importación de calendarios generados por la propia herramienta, y un modelo de costes que demuestra la viabilidad económica de la solución para una base de usuarios de hasta 1.000 usuarios activos por menos de 10 € al mes.

Los resultados obtenidos demuestran que es posible construir una solución de integración de calendarios multi-proveedor funcional, escalable y económicamente viable sobre una arquitectura *serverless* con despliegue automatizado, cumpliendo todos los objetivos planteados en el plan de trabajo.

Palabras clave: iCalendar, CalDAV, WebCal, Google Calendar API, OAuth 2.0, AWS Lambda, *serverless*, Terraform, Angular, CloudFront, fusión de calendarios.

Abstract

Time management through cloud-based calendar services has become a standard practice in both personal and professional contexts. However, when a user works with multiple accounts or providers —Google Calendar, Apple iCloud or any CalDAV-compatible server— achieving a unified view of all their events is particularly challenging in web browser desktop environments, where each tab is typically bound to a single user session.

1

This Bachelor's Thesis addresses this problem through the design and implementation of CalendarFusion, a web-based tool that integrates calendars from different accounts and services, applies filters and transformations to events —such as title prefixes and suffixes, or keyword-based exclusion—, and exposes the result in ICS/WebCal format so that it can be subscribed from Google Calendar, Microsoft Outlook or Apple Calendar.

The solution architecture is fully serverless on AWS: the backend is a Java/Spring Boot microservice packaged as an AWS Lambda function invoked through API Gateway; configuration persistence is handled by DynamoDB; generated ICS file caching uses S3; and periodic refresh of merged calendars is triggered by EventBridge rules. The user interface is a Single Page Application developed with Angular, deployed to S3 and distributed globally via CloudFront with HTTPS. The entire infrastructure is declaratively defined using Terraform, enabling full reproduction with a single command.

OAuth 2.0 authentication support has been implemented for Google Calendar API, as well as support for any CalDAV-compatible server, including Apple iCloud. The tool incorporates circularity prevention logic to avoid importing calendars generated by the tool itself, and a cost model demonstrating the economic viability of the solution for up to 1,000 active users for less than €10 per month.

16

The results demonstrate that it is possible to build a functional, scalable, and economically viable multi-provider calendar integration solution on a serverless architecture with automated deployment, meeting all objectives set out in the work plan.

Keywords: iCalendar, CalDAV, WebCal, Google Calendar API, OAuth 2.0, AWS Lambda, serverless, Terraform, Angular, CloudFront, calendar merging.

Tabla de contenidos

1. Introducción.....	6
1.1 Contexto y motivación.....	6
1.2 Objetivos.....	6
2. Estado del Arte.....	8
2.1 Estándares de representación e intercambio.....	8
2.2 APIs de acceso de los principales proveedores.....	9
2.4 Conclusiones del Estado del Arte.....	11
3. Diseño de la Arquitectura.....	12
3.1 Visión general del sistema.....	12
3.2 Componentes del sistema.....	13
3.2.1 Frontend: SPA Angular.....	13
3.2.2 Backend: AWS Lambda + Spring Boot.....	13
3.2.3 API Gateway HTTP.....	14
3.2.4 Persistencia: DynamoDB.....	14
3.2.5 Caché de ICS: S3.....	15
3.2.6 Refresco automático: EventBridge.....	15
3.2.7 Secrets Manager.....	15
3.2.8 Autenticación: Amazon Cognito.....	15
3.3 Modelo de datos.....	15
3.4 Decisiones de diseño.....	16
3.4.1 Arquitectura serverless vs. servidor dedicado.....	16
3.4.2 Java/Spring Boot en Lambda.....	16
3.4.3 DynamoDB vs. RDS.....	16
4. Implementación del Backend.....	17
4.1 Estructura del proyecto.....	17
4.2 API REST.....	17
4.3 Motor de fusión.....	17
4.3.1 Flujo de fusión.....	18
4.3.2 Integración con Google Calendar API.....	18
4.3.3 Integración con CalDAV.....	20
4.3.4 Filtros y transformaciones.....	21
4.3.5 Prevención de circularidad.....	21
4.3.6 Generación del fichero ICS.....	22
4.4 Tests unitarios del backend.....	22
5. Implementación del Frontend.....	23

5.1	Estructura de la aplicación	23
5.2	Flujo de usuario	24
5.3	Despliegue del frontend	27
5.4	Tests e2e del frontend	27
6.	Infraestructura y Despliegue Automatizado	29
6.1	Infraestructura como Código con Terraform	29
6.2	Distribución con CloudFront	30
6.3	Proceso de despliegue	30
7.	Análisis de Costes AWS	32
7.1	Supuestos del modelo de costes	32
7.2	Costes por servicio	32
7.3	Análisis y conclusiones del modelo de costes	33
8.	Resultados y Conclusiones	35
8.1	Resultados obtenidos	35
8.2	Líneas de trabajo futuro y mejoras	36
8.3	Conclusiones	37
9.	Análisis de Impacto	38
9.1	Impacto personal	38
9.2	Impacto empresarial	38
9.3	Impacto social	38
9.4	Impacto económico	38
9.5	Impacto medioambiental	39
9.6	Impacto cultural	39
10.	Bibliografía	40

Índice de figuras

Figura 1. Diagrama de arquitectura completo de CalendarFusion en AWS: usuario → CloudFront → S3 (SPA Angular)/Amazon Cognito (autenticacion JWT) /API Gateway → Lambda (Spring Boot) → DynamoDB / S3 / Secrets Manager / Google Calendar API / iCloud CalDAV, EventBridge → Lambda (refresco)..... 13

Figura 2. Flujo OAuth2 de integración con Google Calendar API: redirección al endpoint de autorización, callback con código de autorización y obtención del par access_token/refresh_token 19

Figura 3. Llamada al endpoint events.list() de Google Calendar API con los parámetros singleEvents=true y orderBy=startTime para la expansión y ordenación de eventos recurrentes 19

Figura 4. Flujo de descubrimiento CalDAV: secuencia de peticiones PROPFIND para obtener el principal del usuario, el home set de calendarios y la lista de calendarios disponibles 20

Figura 5. Pantalla de inicio de sesión de CalendarFusion 24

Figura 6. Captura del Dashboard de CalendarFusion mostrando la lista de fusiones creadas con sus URLs ICS..... 25

Figura 7. Captura del formulario de Nueva Fusión mostrando la selección de proveedor, calendario, prefijo, sufijo y palabras clave a excluir 26

Figura 8. Captura de la pantalla de gestión de Cuentas mostrando los botones de conexión de Google Calendar y CalDAV 27

Figura 9. Captura de la consola de AWS mostrando la distribución CloudFront con su dominio y estado Deployed..... 30

Figura 10. Desglose de costes reales en AWS – Mayo 2026. AWS Secrets Manager representa el mayor coste fijo (0,30\$), Amazon S3 el mayor coste variable (0,07\$) mientras que DynamoDB, API Gateway y Lambda registran coste 0 al mantenerse dentro del nivel gratuito de AWS 34

Figura 11. Captura del ICS generado en el navegador mostrando eventos de Google Calendar y Apple iCloud fusionados con prefijos distintos..... 35

1

1 Introducción

1.1 Contexto y motivación

10

En la actualidad, la gestión del tiempo y la planificación de actividades se apoyan de forma generalizada en servicios de calendario en la nube. Los principales proveedores Google, Microsoft y Apple, permiten a sus usuarios crear y compartir múltiples calendarios, consumirlos desde distintos dispositivos y combinar en una misma vista eventos procedentes de varias cuentas. Esta capacidad de agregación resulta especialmente cómoda en entornos móviles, donde las aplicaciones nativas integran sin esfuerzo calendarios de distintas cuentas y proveedores.

Sin embargo, en entornos de escritorio basados en navegador web, las posibilidades de visualizar simultáneamente calendarios de diferentes cuentas de un mismo proveedor, o de distintos proveedores, son muy limitadas. Esto se debe en gran medida a que cada pestaña del navegador suele estar vinculada a una única sesión de usuario. Esta situación dificulta la obtención de una visión unificada de la planificación personal u organizativa cuando se trabaja con múltiples cuentas y servicios de calendario.

Si bien algunos proveedores ofrecen mecanismos puntuales para compartir calendarios entre cuentas, estas soluciones no dan respuesta a escenarios más complejos, como la fusión de calendarios de distintos proveedores, la aplicación de filtros y transformaciones sobre los eventos, o la exposición del resultado en un formato estándar consumible desde cualquier plataforma.

A pesar de la existencia de estándares consolidados como iCalendar, CalDAV y WebCal, y de APIs bien documentadas como Google Calendar API, no se dispone de herramientas generalistas que permitan fusionar de forma flexible calendarios procedentes de diferentes cuentas y servicios, aplicar filtros y transformaciones sobre los eventos, y exponer el resultado en formatos estándar para su consumo por las propias plataformas. Esta carencia constituye la motivación principal del presente Trabajo de Fin de Grado.

4

1.2 Objetivos

El objetivo principal de este TFG es diseñar e implementar una herramienta web que permita fusionar calendarios procedentes de diferentes cuentas y servicios en uno nuevo, aplicando filtros y transformaciones sobre los eventos, y ofrecer el resultado en formato ICS/WebCal para su consumo desde plataformas como Google Calendar, Outlook o Apple Calendar. Los objetivos específicos son:

- Integrar calendarios procedentes de múltiples cuentas de un mismo servicio, así como de diferentes servicios, mediante los estándares

iCalendar y CalDAV, e implementar al menos una integración directa con OAuth2 utilizando la Google Calendar API.

- Permitir, de forma visual, fusionar varios calendarios en uno nuevo, aplicando filtros y operaciones de edición sobre los eventos prefijos o sufijos en títulos, exclusión por categorías, políticas de refresco configurables y exponer el resultado en formato ICS/WebCal.
- Implementar el backend con arquitectura serverless en AWS (Lambda, API Gateway, DynamoDB, S3) y desarrollar una SPA con Angular desplegada en S3 y distribuida mediante CloudFront.
- Definir de forma declarativa con Terraform todos los recursos de infraestructura AWS e integrar un pipeline de despliegue automatizado (CI/CD).
- Analizar los costes de la infraestructura en AWS en función del número de usuarios y calendarios gestionados.

2 Estado del Arte

En este capítulo se revisan los estándares, protocolos y APIs sobre los que se apoya el presente trabajo, así como las herramientas existentes en el mercado que abordan problemáticas similares. El objetivo es contextualizar las decisiones de diseño adoptadas y justificar la necesidad de la solución propuesta.

2.1 Estándares de representación e intercambio

Los calendarios digitales se fundamentan en dos estándares abiertos complementarios: iCalendar para la representación de datos y CalDAV para el acceso remoto a esos datos. Ambos son ampliamente adoptados por los principales proveedores. Adicionalmente, WebCal define un mecanismo de suscripción a calendarios publicados en la web, distinto en naturaleza a los anteriores.

iCalendar (RFC 5545) — Representación de datos

iCalendar es el formato estándar para la representación e intercambio de datos de calendario, definido en la RFC 5545 [1]. Su representación es textual y legible por humanos, basada en pares clave:valor organizados en bloques delimitados por etiquetas BEGIN y END. El objeto raíz es VCALENDAR, que contiene componentes como VEVENT (evento), VTODO (tarea) o VJOURNAL (entrada de diario). Cada evento se define mediante propiedades como DTSTART (fecha de inicio), DTEND (fecha de fin), SUMMARY (título), DESCRIPTION (descripción) o RRULE (regla de recurrencia para eventos repetitivos). A continuación se muestra un ejemplo mínimo de fichero ICS con un evento:

```
BEGIN:VCALENDAR
VERSION:2.0
BEGIN:VEVENT
UID:reunion-001@ejemplo.com
DTSTART:20260419T100000Z
DTEND:20260419T110000Z
SUMMARY:[TRABAJO] Reunión de equipo
END:VEVENT
END:VCALENDAR
```

En el ejemplo se aprecia cómo el título del evento incluye el prefijo [TRABAJO], una de las transformaciones que implementará este trabajo. El formato es ampliamente soportado por Google Calendar, Microsoft Outlook y Apple Calendar, lo que lo convierte en el mecanismo de interoperabilidad elegido para exponer el calendario fusionado resultante.

CalDAV (RFC 4791) — Acceso remoto y operaciones

CalDAV es un protocolo de acceso remoto a calendarios basado en WebDAV y HTTP, definido en la RFC 4791 [2]. Mientras iCalendar define cómo se representan los datos, CalDAV define cómo se accede a ellos y qué operaciones

se pueden realizar. Permite a los clientes listar calendarios disponibles, crear, leer, modificar y eliminar eventos (objetos iCalendar) almacenados en un servidor remoto mediante peticiones HTTP estándar (GET, PUT, DELETE) y métodos extendidos de WebDAV (PROPFIND, REPORT). La autenticación se realiza habitualmente mediante Basic Auth sobre HTTPS o mediante tokens. Su principal ventaja es que al tratarse de un protocolo estándar abierto, cualquier cliente compatible puede acceder a servidores de distintos proveedores sin necesidad de una API propietaria. En este trabajo, CalDAV se utiliza como mecanismo genérico de importación de calendarios, permitiendo la integración con Apple iCloud y cualquier otro servidor compatible.

WebCal — Suscripción a calendarios publicados

WebCal es un mecanismo distinto en naturaleza a iCalendar y CalDAV. No define un formato de datos ni un protocolo de acceso bidireccional, sino un esquema de URL (webcal://) que permite suscribirse a un calendario publicado en la web en formato ICS. El cliente descarga periódicamente el fichero ICS de la URL indicada y actualiza su vista local. Es, por tanto, un mecanismo de solo lectura y de actualización periódica. Este mecanismo es precisamente el que utiliza el presente trabajo para exponer el calendario fusionado resultante: la herramienta genera una URL pública con el ICS fusionado que el usuario puede suscribir desde Google Calendar, Outlook o Apple Calendar, de forma que cualquier cambio en los calendarios de origen se propaga automáticamente.

2.2 APIs de acceso de los principales proveedores

Sobre los estándares anteriores, cada proveedor construye sus propias interfaces programáticas. A continuación se describen las tres relevantes para este trabajo, indicando explícitamente el papel que desempeña cada una: Google Calendar API como integración directa de origen, CalDAV como protocolo genérico de origen, y Microsoft Graph como referencia de futuras extensiones.

Google Calendar API

Google Calendar API v3 [3] es la interfaz programática que Google ofrece para acceder y gestionar calendarios de cuentas de Google. Permite listar calendarios, crear, leer, modificar y eliminar eventos, gestionar listas de calendarios y suscribirse a notificaciones de cambios mediante webhooks. La autenticación se realiza mediante el protocolo OAuth 2.0 [4], que permite a una aplicación de terceros acceder a los recursos de un usuario de Google con su consentimiento explícito, sin que la aplicación tenga acceso a sus credenciales. Es el caso de integración directa principal de este trabajo.

Microsoft Graph: Microsoft Graph [5] es la API unificada de Microsoft para acceder a datos de Microsoft 365, incluyendo los calendarios de Outlook. Ofrece endpoints REST para la gestión completa de calendarios y eventos, con autenticación también basada en OAuth 2.0 mediante la plataforma de identidad de Microsoft. Aunque su integración directa queda fuera del alcance de este trabajo, la arquitectura modular diseñada permite incorporarla en el futuro como un nuevo proveedor de origen de calendarios.

CalDAV en Apple iCloud: Apple no ofrece una API propietaria con SDK para el acceso a calendarios de iCloud. En su lugar, expone sus calendarios mediante el protocolo CalDAV estándar, accesible en el servidor `caldav.icloud.com`. La autenticación se realiza mediante contraseñas específicas de aplicación generadas desde el portal de Apple ID, dado que Apple no permite el uso de credenciales principales en aplicaciones de terceros por razones de seguridad. Este mecanismo permite la integración con iCloud Calendar a través del soporte CalDAV genérico implementado en este trabajo.

Herramientas existentes y limitaciones

En este apartado se analizan las principales soluciones existentes que abordan, total o parcialmente, el problema de la visualización e integración de calendarios de distintas cuentas y servicios. Se distinguen tres categorías: soluciones nativas de los propios proveedores, aplicaciones de terceros y herramientas de automatización.

Soluciones nativas de los proveedores

Google Calendar permite visualizar varios calendarios de una misma cuenta en una vista unificada y suscribirse a calendarios externos mediante URLs WebCal, pero no permite integrar en una misma vista web calendarios de cuentas de Google distintas sin cambiar de sesión, ni aplicar filtros o transformaciones sobre los eventos combinados.

Microsoft Outlook, tanto en su versión web como de escritorio, admite añadir cuentas de diferentes proveedores (Gmail, iCloud) y visualizarlas conjuntamente como se puede apreciar en su interfaz web, pero no genera calendarios fusionados exportables ni permite aplicar reglas de transformación.

Apple Calendar permite suscribirse a calendarios externos y visualizar calendarios de iCloud y CalDAV, pero su integración con cuentas de Google o Microsoft es limitada en entornos web. En todos los casos, las soluciones nativas están diseñadas para la visualización local, no para la generación de calendarios fusionados exportables con transformaciones configurables.

Aplicaciones de terceros

Fantastical [6] es una aplicación nativa para macOS e iOS que agrega en una sola vista calendarios de Google, iCloud, Outlook y Exchange, con soporte para eventos de videoconferencia y una interfaz muy cuidada. Calendars 5 [7] es otra aplicación nativa para iOS orientada a dispositivos móviles que agrega calendarios de Google y iCloud. Ambas aplicaciones están limitadas a entornos nativos Apple y no permiten exportar el resultado fusionado como una URL ICS/WebCal consumible desde otras plataformas. Merge Calendar [8] es una herramienta web específica de fusión de calendarios ICS que combina varias URLs ICS/WebCal públicas en una sola URL de salida, pero no soporta integraciones directas con APIs de proveedores mediante OAuth2, ni ofrece filtros avanzados, transformaciones de eventos ni políticas de refresco configurables.

Herramientas de automatización

Zapier[9] y Make (anteriormente Integromat) son plataformas de automatización que permiten conectar servicios de calendario con otras aplicaciones, como copiar eventos de un calendario a otro o disparar acciones al crear un evento. Sin embargo, no están orientadas a la fusión de calendarios en una URL WebCal unificada, requieren configuraciones complejas por caso de uso y no resuelven la propagación automática de cambios ni la prevención de circularidades.

2.4 Conclusiones del Estado del Arte

Del análisis realizado se extraen las siguientes conclusiones. Los estándares iCalendar y CalDAV proporcionan una base sólida e interoperable para la representación y el acceso a calendarios, y son ampliamente soportados por los principales proveedores. Sin embargo, la existencia de estos estándares no ha derivado en herramientas generalistas que aprovechen su potencial para fusionar calendarios de forma flexible.

Las soluciones nativas de Google, Microsoft y Apple ofrecen capacidades de visualización conjunta, pero dentro de sus propios ecosistemas y sin posibilidad de exportar el resultado como un calendario estándar con transformaciones aplicadas. Las aplicaciones de terceros como Fantastical o Calendars 5 mejoran la experiencia en entornos nativos móviles, pero tampoco resuelven el problema en entornos web ni ofrecen capacidades de transformación y exportación. Herramientas como Merge Calendar abordan la fusión de ICS públicos, pero sin integración con APIs autenticadas. Las plataformas de automatización como Zapier son demasiado genéricas y no resuelven el caso de uso de forma directa.

En consecuencia, no existe actualmente ninguna herramienta que combine de forma integrada: (1) integración directa con APIs de proveedores mediante OAuth2, (2) soporte genérico a CalDAV, (3) aplicación de filtros y transformaciones configurables sobre los eventos, (4) gestión de políticas de refresco y prevención de circularidades, y (5) exposición del resultado en formato ICS/WebCal mediante una arquitectura *serverless* escalable con despliegue automatizado. Esta carencia justifica y motiva el desarrollo del presente Trabajo de Fin de Grado.

1

3 Diseño de la Arquitectura

En este capítulo se describe el diseño arquitectónico de CalendarFusion, desde la visión global del sistema hasta las decisiones de diseño adoptadas para cada componente. El objetivo es justificar por qué se han elegido las tecnologías y patrones empleados, y describir cómo se integran entre sí para cumplir los requisitos funcionales y no funcionales del trabajo.

3.1 Visión general del sistema

CalendarFusion sigue una arquitectura cliente-servidor desacoplada, en la que el *frontend* una Single Page Application y el *backend* —un conjunto de funciones *serverless*— se comunican exclusivamente a través de una API REST. Esta separación permite escalar y evolucionar ambas capas de forma independiente.

El diagrama siguiente muestra la arquitectura completa del sistema desplegada en AWS:

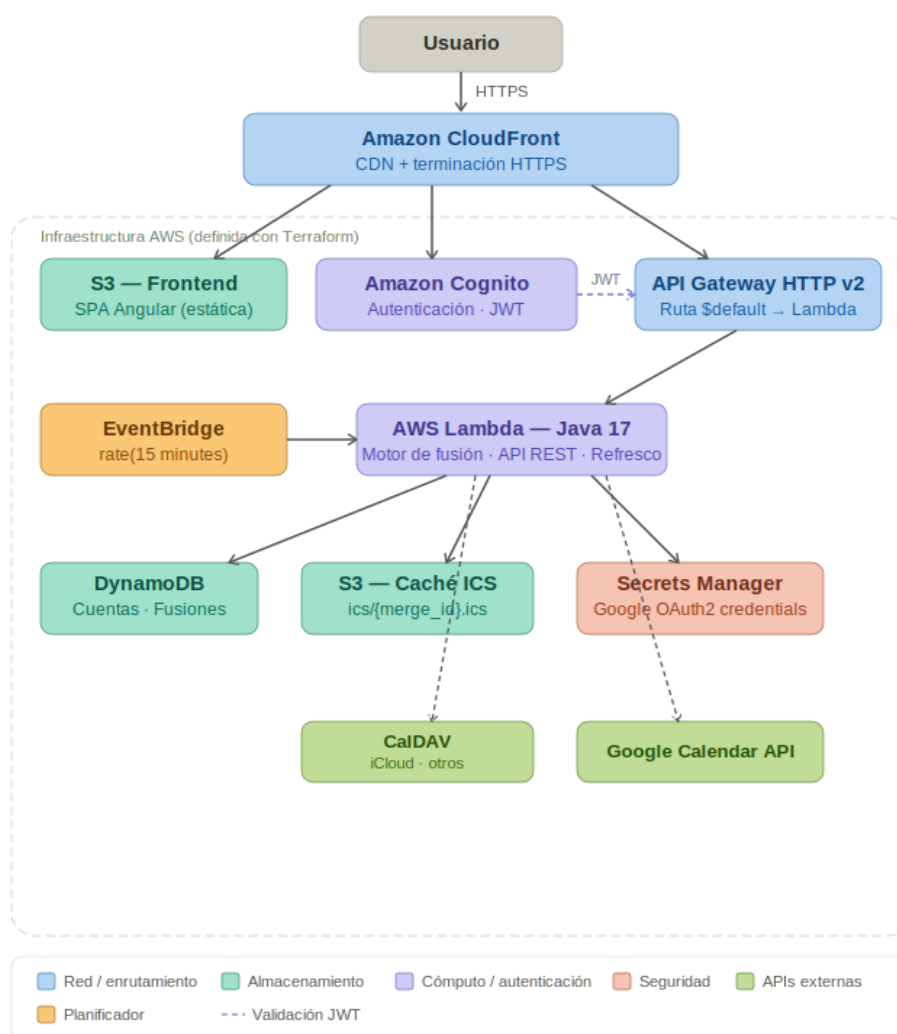


Figura 1. Diagrama de arquitectura completo de CalendarFusion en AWS: usuario → CloudFront → S3 (SPA Angular) / Amazon Cognito (autenticación JWT) / API Gateway → Lambda (Spring Boot) → DynamoDB / S3 / Secrets Manager / Google Calendar API / iCloud CalDAV, EventBridge → Lambda (refresco)

El flujo principal de uso es el siguiente. El usuario accede a la SPA Angular a través de la URL HTTPS de CloudFront. Desde la interfaz puede conectar sus cuentas de Google Calendar mediante OAuth 2.0 o cualquier servidor CalDAV mediante credenciales Basic Auth. Una vez conectadas las cuentas, define una configuración de fusión especificando los calendarios de origen, los filtros y transformaciones a aplicar, y la política de refresco. El *backend* genera un identificador único de fusión (*merge_id*) y devuelve una URL ICS que el usuario puede suscribir desde cualquier plataforma de calendario compatible. Las fusiones se refrescan automáticamente cada 15 minutos mediante un trigger de EventBridge, que invoca el Lambda y actualiza el fichero ICS en S3.

3.2 Componentes del sistema

3.2.1 Frontend: SPA Angular

La interfaz de usuario es una Single Page Application (SPA) desarrollada con Angular 17. La elección de Angular se justifica por su arquitectura basada en componentes, su sistema de enrutamiento cliente-lado que permite navegar entre vistas sin recarga de página, y su sistema de inyección de dependencias que facilita la separación de responsabilidades entre componentes y servicios.

La aplicación se organiza en cuatro rutas principales: el panel de control (*Dashboard*), que lista las fusiones existentes; la gestión de Cuentas, que permite conectar cuentas de Google y CalDAV; el formulario de Nueva Fusión, donde se configura cada fusión; y el formulario de Conexión CalDAV. La comunicación con el *backend* se centraliza en servicios Angular que encapsulan las llamadas HTTP a la API REST.

La aplicación se despliega como un conjunto de ficheros estáticos en un bucket S3 y se distribuye globalmente mediante Amazon CloudFront, que actúa como CDN y termina las conexiones HTTPS. CloudFront está configurado para redirigir cualquier ruta desconocida a *index.html*, permitiendo que el enrutador de Angular gestione la navegación en cliente.

3.2.2 Backend: AWS Lambda + Spring Boot

El *backend* es un microservicio Java 17 desarrollado con Spring Boot 3.2, empaquetado como un JAR ejecutable en AWS Lambda mediante la librería

aws-serverless-java-container. Esta librería actúa como adaptador entre el modelo de invocación de Lambda y el modelo de Servlet de Spring MVC, permitiendo reutilizar el código Spring Boot estándar sin modificaciones.

La función Lambda es un único despliegue que gestiona todos los endpoints de la API. El *handler* (el manejador) principal (*Main.java*) detecta si la invocación proviene de una petición HTTP (API Gateway) o de un evento programado de EventBridge, y delega en el *handler* correspondiente:

```
public void handleRequest(InputStream input, OutputStream output,
                          Context context) throws IOException {
    JsonNode event = mapper.readTree(input.readAllBytes());
    if ("aws.events".equals(event.path("source").asText())) {
        ScheduledRefreshService.getInstance().refreshAll();
        output.write("{\"status\":\"ok\"}".getBytes());
        return;
    }
    handler.proxyStream(new ByteArrayInputStream(bytes), output, context);
}
```

La memoria asignada a la función es de 512 MB, con un *timeout* de 30 segundos por invocación. El *cold start* típico es de 5-6 segundos, siendo el arranque de Spring Boot el factor dominante. Las invocaciones subsiguientes en la misma instancia se resuelven en menos de 500 ms.

3.2.3 API Gateway HTTP

Amazon API Gateway v2 (HTTP API) actúa como punto de entrada para todas las peticiones al *backend*. Se ha utilizado la variante HTTP API en lugar de REST API por su menor latencia y coste. La integración con Lambda es de tipo AWS_PROXY, lo que significa que API Gateway reenvía la petición completa al Lambda sin transformación, y Lambda devuelve la respuesta completa incluyendo cabeceras y código de estado.

Se ha configurado una ruta comodín (*\$default*) que captura todas las rutas y las delega a Spring Boot, evitando la necesidad de definir una ruta por punto de acceso (*endpoint*) en API Gateway. El CORS se configura directamente en API Gateway, permitiendo el acceso desde la URL de CloudFront y desde *localhost* para desarrollo.

3.2.4 Persistencia: DynamoDB

La persistencia del sistema se gestiona mediante dos tablas de Amazon DynamoDB con esquema de clave simple:

- *tfg-calendar-accounts*: almacena las credenciales de las cuentas conectadas. La clave primaria es *account_id* (String). Cada *ítem* contiene el proveedor (google o caldav), el credencial de acceso (*token*) de refresco OAuth2 o las credenciales CalDAV, y la fecha de última actualización.
- *tfg-merge-configs*: almacena las configuraciones de fusión. La clave primaria es *merge_id* (String). Cada *ítem* contiene la configuración completa serializada como JSON, incluyendo las fuentes, filtros y parámetros de refresco.

Se ha elegido DynamoDB por su modelo de facturación por demanda (PAY_PER_REQUEST), que elimina la necesidad de aprovisionar capacidad y

resulta económico para cargas variables como las de este proyecto. El acceso se realiza exclusivamente mediante el SDK de AWS para Java v2.

3.2.5 Caché de ICS: S3

Los ficheros ICS generados por el motor de fusión se almacenan en Amazon S3 bajo la clave `ics/{merge_id}.ics`. Cuando un cliente solicita el ICS de una fusión, el Lambda intenta servirlo desde la caché S3. Si el fichero no existe o ha caducado, genera el ICS en tiempo real fusionando las fuentes, lo sirve y lo almacena en S3 para futuras peticiones. Este mecanismo reduce significativamente la latencia de las peticiones de suscripción recurrentes.

3.2.6 Refresco automático: EventBridge

Amazon EventBridge actúa como planificador de tareas. Se ha definido una regla con expresión `rate(15 minutes)` que invoca la función Lambda cada 15 minutos. El Lambda detecta este tipo de invocación por la presencia del campo `"source": "aws.events"` en el evento, y delega en `ScheduledRefreshService`, que recupera todas las configuraciones de fusión de DynamoDB y regenera sus ficheros ICS en S3.

3.2.7 Secrets Manager

Las credenciales OAuth2 de Google (`client_id`, `client_secret` y `scopes`) se almacenan en AWS Secrets Manager, evitando su inclusión en el código fuente o en variables de entorno en texto claro. El Lambda recupera el secreto en el momento de la inicialización y lo utiliza para construir el flujo de autenticación OAuth2.

3.2.8 Autenticación: Amazon Cognito

La autenticación y gestión de usuarios se realiza mediante Amazon Cognito, que proporciona registro, inicio de sesión y gestión de tokens JWT para los usuarios de la aplicación. Cognito actúa como proveedor de identidad, emitiendo tokens de acceso que el *frontend* Angular adjunta en las cabeceras de cada petición HTTP al *backend*. API Gateway valida estos tokens mediante un autorizador JWT antes de invocar la función Lambda, garantizando que únicamente los usuarios autenticados pueden acceder a los recursos protegidos de la API.

3.3 Modelo de datos

El sistema maneja tres entidades principales: la cuenta de calendario, la fuente de fusión y la configuración de fusión.

La `MergeSource` representa un calendario de origen dentro de una fusión y contiene los siguientes campos:

- `provider`: tipo de proveedor ("google" o "caldav")
- `accountId`: identificador de la cuenta conectada

- `calendarId`: identificador del calendario dentro del proveedor
- `prefix`: prefijo opcional a añadir al título de cada evento
- `suffix`: sufijo opcional a añadir al título de cada evento
- `excludeKeywords`: lista de palabras clave cuyos eventos se excluirán del resultado

La `MergeConfig` representa la configuración completa de una fusión y contiene: el identificador único de la fusión (`mergeId`), la lista de fuentes (`sources`) y el número máximo de eventos por calendario (`maxResultsPerCalendar`).

3.4 Decisiones de diseño

3.4.1 Arquitectura *serverless* vs. servidor dedicado

La elección de una arquitectura *serverless* sobre un servidor dedicado o containerizado se justifica por tres razones principales. En primer lugar, el modelo de costes por invocación se ajusta perfectamente a la naturaleza del proyecto: el tráfico es muy irregular, mayoritariamente peticiones periódicas de suscripción ICS, y no justifica mantener una instancia activa permanentemente. En segundo lugar, AWS Lambda gestiona automáticamente el escalado, eliminando la necesidad de configurar grupos de autoescalado. En tercer lugar, la ausencia de gestión de servidor simplifica el mantenimiento operativo del sistema.

3.4.2 Java/Spring Boot en Lambda

La elección de Java/Spring Boot para el *backend*, en lugar de un entorno de ejecución (*runtime*) más ligero como Node.js o Python, se justifica por la disponibilidad de librerías maduras para el acceso a las APIs de calendario (Google Calendar API Java Client, ical4j, Apache HttpClient) y por la familiaridad del autor con el ecosistema. El principal inconveniente es el arranque en frío (*cold start*) de 5-6 segundos, que se mitiga mediante el refresco automático periódico con EventBridge, que mantiene la instancia Lambda caliente, y por la naturaleza no interactiva de las peticiones de suscripción ICS.

3.4.3 DynamoDB vs. RDS

Se ha elegido DynamoDB sobre una base de datos relacional por dos razones. El modelo de datos del proyecto es fundamentalmente un mapa de clave-valor (`account_id` → credenciales, `merge_id` → configuración JSON), que encaja naturalmente con el modelo de DynamoDB. Además, el modo `PAY_PER_REQUEST` de DynamoDB elimina el coste fijo de mantener una instancia RDS activa, que para el volumen esperado representaría el grueso del coste mensual.

4 Implementación del Backend

En este capítulo se describe la implementación del *backend* de CalendarFusion, detallando los componentes principales, los algoritmos implementados y las decisiones técnicas relevantes.

4.1 Estructura del proyecto

El *backend* se organiza como un proyecto Maven estándar de Spring Boot bajo el directorio

aws-lambda-01/. Los paquetes principales son:

- tfg.prod.controller: controladores REST (MergeController, CalDavController, GoogleCalendarController)
- tfg.prod.services: lógica de negocio (CalendarMergeService, IcsGeneratorService, DynamoDbService, GoogleTokenService, CalDavSessionService, ScheduledRefreshService)
- tfg.prod.modules: clases de modelo (MergeConfig, MergeSource, MergedEvent)

4.2 API REST

La API expone los siguientes endpoints principales:

Método	Ruta	Descripción
POST	/caldav/auth/login	Conectar cuenta CalDAV (Basic Auth)
GET	/caldav/{accountId}/calendars	Listar calendarios de cuenta CalDAV
GET	/google-calendar/auth/google	Iniciar flujo OAuth2 con Google
GET	/google-calendar/auth/google/callback	respuesta de retorno (<i>Callback</i>) OAuth2 de Google
GET	/google-calendar/{accountId}/calendars	Listar calendarios de cuenta Google
POST	/merge	Crear configuración de fusión
GET	/merge	Listar todas las fusiones
GET	/merge/{mergeId}/ics	Obtener ICS generado de una fusión
DELETE	/merge/{mergeId}	Eliminar una fusión

Tabla 1: Endpoints de la API REST de CalendarFusion.

4.3 Motor de fusión

El motor de fusión es el componente central del sistema. Se implementa en la clase

CalendarMergeService y es responsable de obtener los eventos de cada fuente, aplicar los filtros y transformaciones configurados, y devolver la lista unificada de eventos fusionados.

4.3.1 Flujo de fusión

El método `merge(MergeConfig config)` itera sobre la lista de fuentes y delega la obtención de eventos al método correspondiente según el proveedor:

```
public List<MergedEvent> merge(MergeConfig config) {
    List<MergedEvent> all = new ArrayList<>();
    for (MergeSource source : config.getSources()) {
        try {
            if ("google".equals(source.getProvider()))
                all.addAll(fetchGoogle(source,
                    config.getMaxResultsPerCalendar()));
            else if ("caldav".equals(source.getProvider()))
                all.addAll(fetchCalDav(source));
        } catch (Exception e) {
            // Error en una fuente no detiene el proceso
            System.err.println("Error: " + e.getMessage());
        }
    }
    return all;
}
```

El manejo de errores por fuente garantiza que un fallo en una fuente no impide la generación del ICS con los eventos de las fuentes restantes.

4.3.2 Integración con Google Calendar API

La integración con Google Calendar API utiliza la librería oficial Google API Client para Java. El flujo de autenticación OAuth 2.0 sigue el Authorization Code Flow estándar:

- El usuario pulsa 'Conectar con Google' en la UI, que redirige al *endpoint* `/google-calendar/auth/google`.
- El *backend* construye la URL de autorización de Google con los scopes necesarios (`calendar.readonly`) y redirige al usuario.
- Google autentica al usuario y redirige al *callback* `/google-calendar/auth/google/callback` con un código de autorización.
- El *backend* intercambia el código por un par `access_token` / `refresh_token` y almacena el `refresh_token` en DynamoDB.
- En cada fusión, el *backend* recupera el `refresh_token` de DynamoDB y obtiene un `access_token` fresco para realizar las llamadas a la API.

La obtención de eventos utiliza el método `events.list()` de la librería oficial Google API Client para Java, que internamente realiza una petición al *endpoint* de la API REST de Google Calendar. Se emplea el parámetro `singleEvents=true` para expandir eventos recurrentes en instancias individuales, y `orderBy=startTime`.

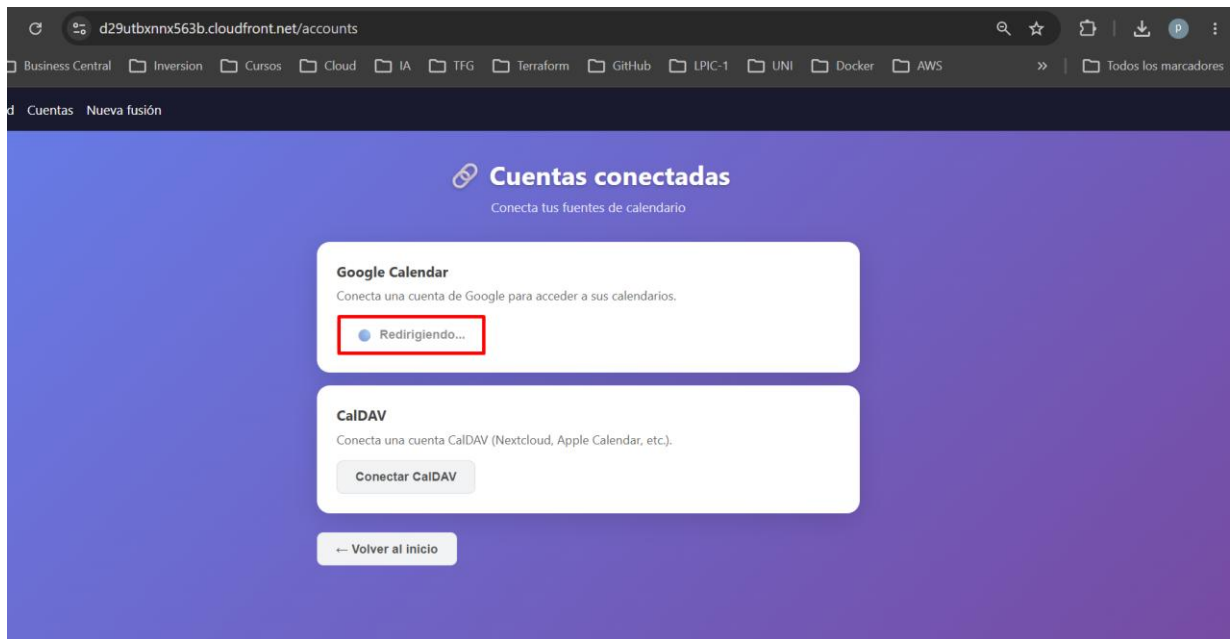


Figura 2. Flujo OAuth2 de integración con Google Calendar API: redirección al endpoint de autorización, callback con código de autorización y obtención del par `access_token/refresh_token`

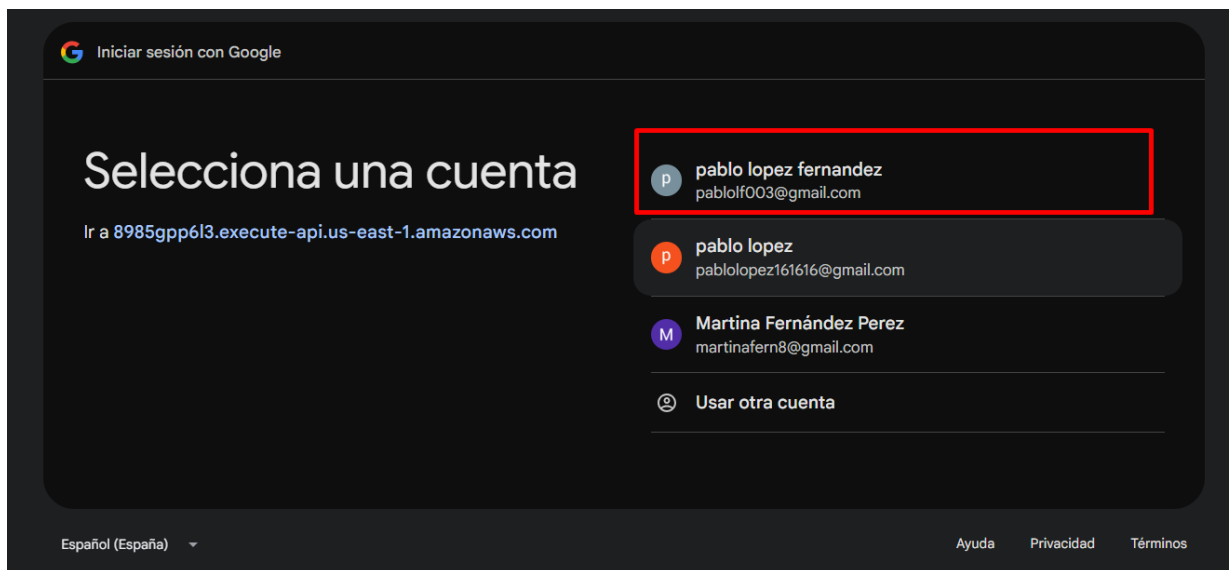


Figura 3. Llamada al endpoint `events.list()` de Google Calendar API con los parámetros `singleEvents=true` y `orderBy=startTime` para la expansión y ordenación de eventos recurrentes

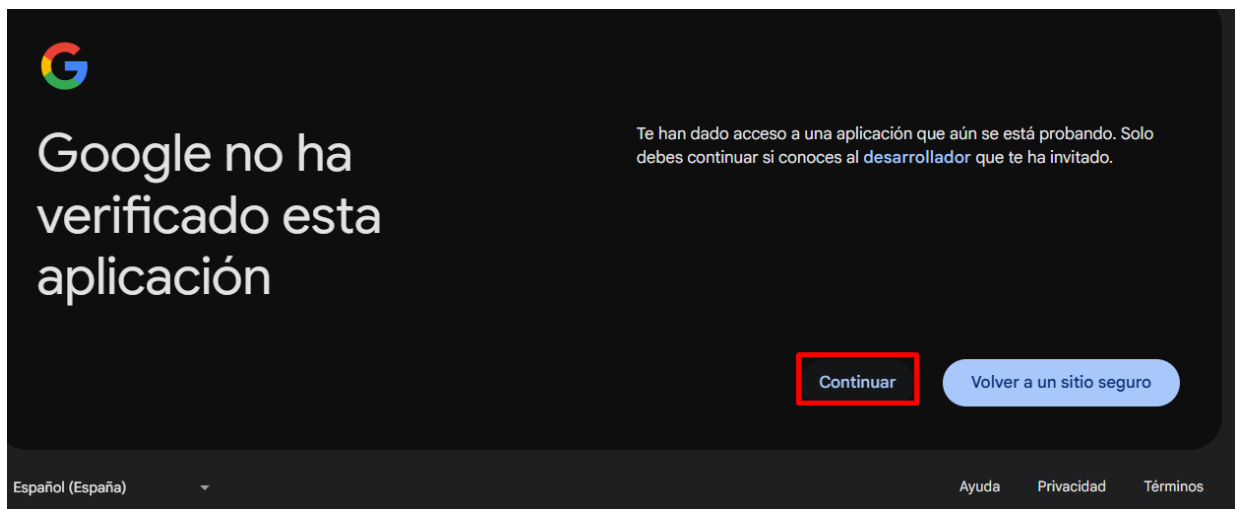


Figura 4. Flujo de descubrimiento CalDAV: secuencia de peticiones PROPFIND para obtener el principal del usuario, el home set de calendarios y la lista de calendarios disponibles

4.3.3 Integración con CalDAV

La integración con servidores CalDAV implementa el protocolo RFC 4791 mediante peticiones HTTP directas con Apache HttpClient 5. El proceso de descubrimiento de calendarios sigue la secuencia definida por el protocolo:

- PROPFIND a la URL raíz del servidor para obtener el principal del usuario (current-user-principal).
- PROPFIND al principal para obtener el home set de calendarios (calendar-home-set).
- PROPFIND al home set para listar los calendarios disponibles con sus propiedades (displayname, resource type).

La obtención de eventos utiliza el método REPORT con una query de tipo calendar-query en formato XML, que permite filtrar los eventos por rango de fechas (time-range):

```
<C:calendar-query xmlns:D="DAV:"
                  xmlns:C="urn:ietf:params:xml:ns:caldav">
  <D:prop><D:getetag/><C:calendar-data/></D:prop>
  <C:filter>
    <C:comp-filter name="VCALENDAR">
      <C:comp-filter name="VEVENT">
        <C:time-range start="20200101T000000Z"
                      end="20271231T235959Z"/>
      </C:comp-filter>
    </C:comp-filter>
  </C:filter>
</C:calendar-query>
```

Una particularidad importante de la respuesta de iCloud CalDAV es que los datos ICS se devuelven con entidades XML escapadas (
 en lugar de saltos de línea reales). El método parseIcs() decodifica estas entidades antes

del parseo, lo que permite la compatibilidad con iCloud y otros servidores CalDAV.

4.3.4 Filtros y transformaciones

Cada fuente puede configurar los siguientes filtros y transformaciones sobre sus eventos:

- Prefijo (prefix): texto que se antepone al título de cada evento. Permite identificar visualmente el origen de los eventos en el calendario fusionado.
- Sufijo (suffix): texto que se añade al final del título de cada evento.
- Exclusión por palabras clave (excludeKeywords): lista de términos que, si aparecen en el título o descripción de un evento, provocan su exclusión del calendario fusionado.

Estas transformaciones se aplican de forma independiente por cada fuente dentro de una misma fusión, lo que permite, por ejemplo, que los eventos de Google Calendar aparezcan con el prefijo [TRABAJO] mientras que los de iCloud lo hagan con [PERSONAL], facilitando su identificación visual en el calendario resultante. La exclusión por palabras clave actúa como filtro previo: si un evento contiene alguno de los términos configurados en su título o descripción, se descarta antes de añadirse al calendario fusionado, independientemente de si tiene prefijo o sufijo configurados.

La lógica de exclusión se implementa en el método `shouldExclude()` de `CalendarMergeService`:

```
private boolean shouldExclude(MergedEvent event, MergeSource source) {
    if (source.getExcludeKeywords() == null ||
        source.getExcludeKeywords().isEmpty()) return false;
    String summary = event.getSummary().toLowerCase();
    String desc    = event.getDescription() != null
        ? event.getDescription().toLowerCase() : "";
    for (String keyword : source.getExcludeKeywords()) {
        if (summary.contains(keyword.toLowerCase()) ||
            desc.contains(keyword.toLowerCase())) return true;
    }
    return false;
}
```

4.3.5 Prevención de circularidad

Un riesgo inherente a las herramientas de fusión de calendarios es la circularidad: si un usuario suscribe el ICS generado por `CalendarFusion` como fuente de otra fusión, los eventos se duplicarían indefinidamente. Para evitar este problema, el método `isCircular()` detecta si el ICS de un evento fue generado por la propia herramienta comprobando el campo `PRODID` del fichero ICS:

```
private boolean isCircular(String icsContent) {
    return icsContent != null &&
        icsContent.contains("PRODID:--/TFG Calendar Merger//EN");
}
```

}

Si se detecta circularidad, el evento se descarta y se registra un mensaje de log. Todos los ICS generados por CalendarFusion incluyen el PRODID "TFG Calendar Merger", lo que actúa como firma digital que permite identificarlos como propios.

4.3.6 Generación del fichero ICS

La clase IcsGeneratorService implementa la generación del fichero ICS resultante conforme al estándar RFC 5545. Cada evento se serializa con sus propiedades obligatorias (UID, DTSTART, DTEND, SUMMARY) y opcionales (DESCRIPTION, LOCATION). Los valores especiales se escapan según el estándar (las comas con \,, los puntos y comas con \; y los saltos de línea con \n).

Los eventos de todo el día se serializan con la propiedad DTSTART;VALUE=DATE en formato YYYYMMDD, mientras que los eventos con hora se serializan con DTSTART en formato YYYYMMDDTHHmss.

4.4 Tests unitarios del backend

El *backend* incluye una suite de tests unitarios implementada con JUnit 5 y Mockito, organizados en dos clases:

- IcsGeneratorServiceTest: 7 tests que verifican la correcta generación del fichero ICS para distintos casos (calendario vacío, eventos de todo el día, eventos con hora, escape de caracteres especiales, múltiples eventos).
- CalendarMergeServiceTest: 8 tests que verifican la lógica de fusión, el parseo de campos ICS, el manejo de errores por fuente, y el comportamiento con proveedores desconocidos.

Los tests son de tipo funcional y verifican que cada método produce el resultado esperado ante distintas entradas, incluyendo casos límite y situaciones de error. Se ejecutan con el comando `mvn test` y no requieren conexión a AWS ni a servicios externos, ya que los servicios dependientes se simulan con Mockito. No se ha configurado una herramienta de medición de cobertura de código como JaCoCo, por lo que no se dispone de una métrica formal de cobertura.

5 Implementación del Frontend

En este capítulo se describe la implementación de la interfaz de usuario de CalendarFusion, una Single Page Application desarrollada con Angular 17.

5.1 Estructura de la aplicación

La aplicación Angular se organiza en los siguientes módulos y componentes principales:

- **LoginComponent**: pantalla de inicio de sesión de la aplicación. Permite al usuario autenticarse mediante Amazon Cognito antes de acceder al resto de la aplicación. Una vez autenticado, el *token* JWT generado por Cognito se incluye en cada petición al *backend*, que lo valida antes de procesar cualquier operación.
- **DashboardComponent**: pantalla principal de la aplicación. Lista todas las fusiones creadas, mostrando el identificador, la URL ICS y botones para copiar la URL y eliminar la fusión.
- **AccountsComponent**: gestión de cuentas conectadas. Permite iniciar el flujo OAuth2 con Google y navegar al formulario de conexión CalDAV.
- **ConnectCaldavComponent**: formulario de conexión de cuentas CalDAV. Solicita el nombre de la cuenta, la URL del servidor, el usuario y la contraseña.
- **MergeNewComponent**: formulario de creación de nuevas fusiones. Permite añadir múltiples fuentes de calendarios, configurar prefijos, sufijos y palabras clave de exclusión, y lanzar la creación de la fusión.

La comunicación con el *backend* se centraliza en tres servicios Angular:

- **MergeService**: operaciones CRUD sobre las fusiones (crear, listar, eliminar, obtener ICS).
- **GoogleCalendarService**: conexión con cuentas de Google y listado de calendarios.
- **CalDavService**: conexión con servidores CalDAV y listado de calendarios.
- **AuthService**: gestión del estado de autenticación del usuario, incluyendo el inicio y cierre de sesión contra Cognito y el almacenamiento del *token* JWT para su uso en las peticiones HTTP.

5.2 Flujo de usuario

A continuación se describe el flujo principal de uso de la aplicación:

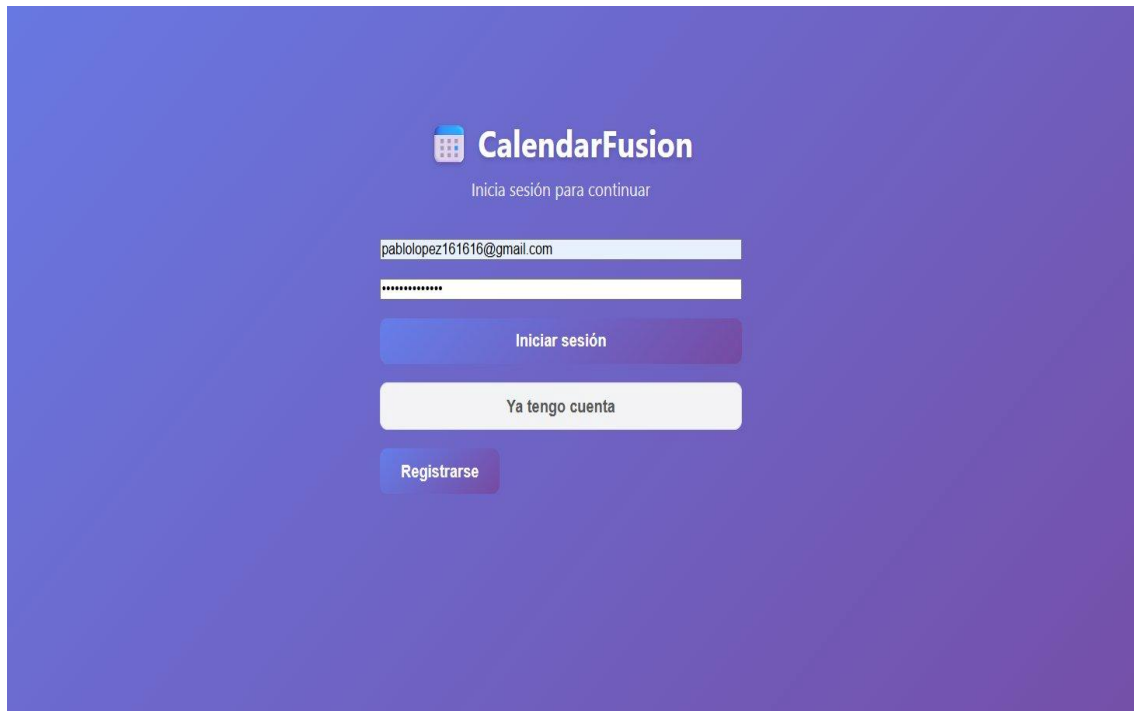


Figura 5. Pantalla de inicio de sesión de CalendarFusion

El usuario inicia sesión/se registra con su correo electrónico y contraseña.

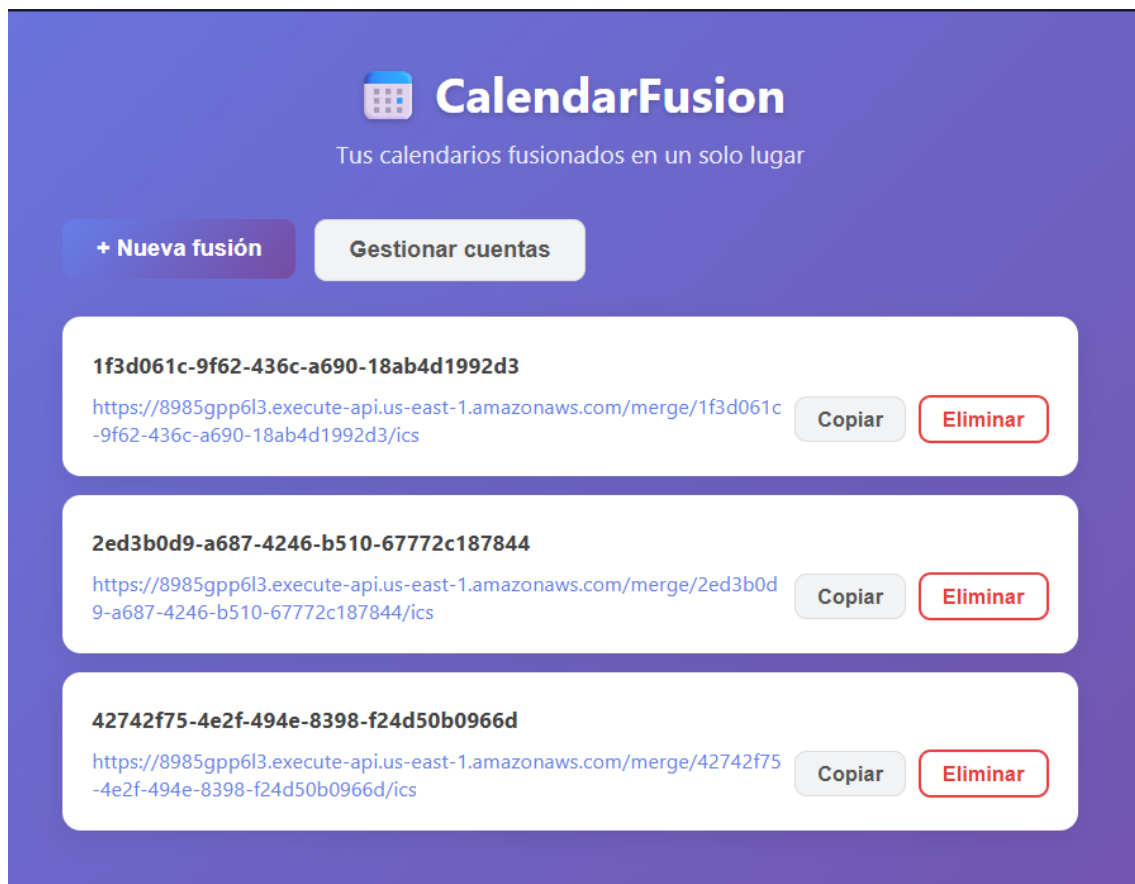


Figura 6. Captura del Dashboard de CalendarFusion mostrando la lista de fusiones creadas con sus URLs ICS

El usuario accede al *Dashboard*, donde puede ver sus fusiones existentes. Para crear una nueva fusión, pulsa '+ Nueva fusión' y accede al formulario de configuración:

Combina varios calendarios en uno

Calendarios fuente

Proveedor
Google Calendar

ID de cuenta
ej: google-1234567890

Calendario
ej: primary Cargar

Prefijo en títulos (opcional)
ej: [Trabajo]

Sufijo en títulos (opcional)
ej: (Personal)

Palabras clave a excluir (separadas por coma)
ej: Cancelar, festivo, cumpleaños

Eliminar fuente

+ Añadir calendario fuente

Figura 7. Captura del formulario de Nueva Fusión mostrando la selección de proveedor, calendario, prefijo, sufijo y palabras clave a excluir

En el formulario de nueva fusión, el usuario puede añadir tantas fuentes como desee. Para cada fuente selecciona el proveedor (Google Calendar o CalDAV), la cuenta conectada, el calendario específico dentro de esa cuenta, y los filtros y transformaciones opcionales. Una vez creada la fusión, la aplicación muestra la URL ICS que puede copiarse y suscribirse desde cualquier plataforma de calendario.

Para cada fuente añadida, el usuario puede configurar de forma independiente prefijos, sufijos y palabras clave de exclusión sobre sus eventos. El detalle de cómo funcionan estas transformaciones se describe en la sección 4.3.4.

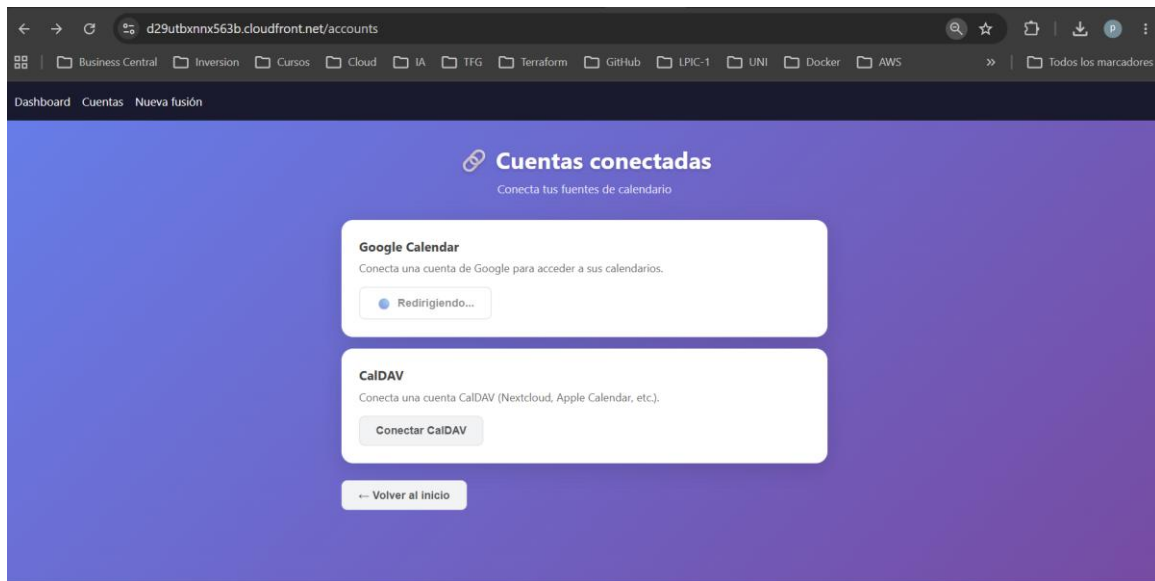


Figura 8. Captura de la pantalla de gestión de Cuentas mostrando los botones de conexión de Google Calendar y CalDAV

5.3 Despliegue del frontend

El proceso de construcción y despliegue del *frontend* está completamente automatizado mediante Terraform. El recurso `null_resource.frontend_deploy` ejecuta el siguiente comando como parte del terraform `apply`:

```
npm install
ng build --configuration production
aws s3 sync dist/angular-frontend/browser/ s3://aws-tfg-frontend-plfz/ --
delete
```

Este proceso compila la aplicación Angular en modo producción (con tree-shaking y minificación), y sincroniza los ficheros resultantes con el bucket S3. CloudFront sirve estos ficheros con el dominio HTTPS de la distribución, y las invalidaciones de caché necesarias tras cada despliegue se realizan manualmente desde la consola de AWS.

5.4 Tests e2e del frontend

El *frontend* incluye una suite de tests end-to-end implementada con Cypress 15. Los tests se organizan en cuatro grupos de prueba:

- *Dashboard* (7 tests): verifican que la lista de fusiones se carga correctamente, que los botones de copiar y eliminar funcionan, y que la navegación hacia los formularios de nueva fusión y gestión de cuentas es correcta.
- *Cuentas* (4 tests): verifican que las tarjetas de Google y CalDAV se muestran, que el botón de conexión con Google es visible, que el parámetro `connected` se procesa correctamente, y que la navegación al formulario CalDAV funciona.

- Conectar CalDAV (3 tests): verifican que el formulario muestra los campos necesarios, que una conexión exitosa redirige a la pantalla de cuentas, y que una conexión fallida muestra el mensaje de error.
- Nueva fusión (3 tests): verifican que el formulario se carga, que la creación de una fusión devuelve la URL ICS, y que el intento de crear sin fuentes muestra el mensaje de validación.

Los tests utilizan la funcionalidad de intercepción de peticiones HTTP de Cypress (`cy.intercept()`) para simular las respuestas del *backend* sin necesidad de conexión real, lo que permite ejecutarlos en cualquier entorno de desarrollo con el *frontend* corriendo localmente.

6 Infraestructura y Despliegue Automatizado

En este capítulo se describe la infraestructura AWS de CalendarFusion y su definición declarativa con Terraform, así como el proceso de despliegue automatizado.

6.1 Infraestructura como Código con Terraform

Toda la infraestructura de CalendarFusion se define de forma declarativa con Terraform, lo que permite reproducir el entorno completo en cualquier cuenta AWS con un único comando. El estado de Terraform se almacena en un bucket S3 (aws-tfg-bucket-plfz), garantizando su persistencia y la posibilidad de trabajo colaborativo.

La infraestructura se organiza en los siguientes bloques lógicos dentro del fichero

main.tf:

- Lambda: definición de la función, configuración de memoria (512 MB) y tiempo de espera (*timeout*) (30 s), variables de entorno, y subida del JAR a S3.
- API Gateway: API HTTP v2 con integración AWS_PROXY, ruta comodín \$default, configuración CORS, y permisos de invocación.
- DynamoDB: dos tablas (tfg-calendar-accounts y tfg-merge-configs) en modo PAY_PER_REQUEST con sus políticas IAM correspondientes.
- *Frontend* S3 + CloudFront: bucket S3 con hosting web estático, distribución CloudFront con HTTPS, redirección de errores 403/404 a index.html, y despliegue automático mediante null_resource.
- Secrets Manager: secreto para las credenciales OAuth2 de Google y política IAM de acceso desde Lambda.
- EventBridge: regla de ejecución periódica (rate 15 minutes), target Lambda, y permiso de invocación.

Un extracto representativo de la definición Terraform de la función Lambda:

```
resource "aws_lambda_function" "aws-lambda-tfg" {
  function_name = "aws-lambda-tfg"
  role          = data.aws_iam_role.lambda_role.arn
  runtime       = "java17"
  handler       = "tfg.prod.Main::handleRequest"
  s3_bucket     = aws_s3_bucket.aws-lambda-tfg-bucket.bucket
  s3_key        = aws_s3_object.lambda_jar.key
  source_code_hash = filebase64sha256(var.lambda_jar_path)
  memory_size   = 512
  timeout       = 30
  environment {
    variables = {
      DYNAMODB_TABLE           = aws_dynamodb_table.calendar_accounts.name
      DYNAMODB_MERGE_TABLE     = aws_dynamodb_table.merge_configs.name
      GOOGLE_CREDENTIALS_SECRET_ARN = ...
      FRONTEND_URL             = "https://${cloudfront.domain_name}"
      ICS_CACHE_BUCKET         = aws_s3_bucket.aws-lambda-tfg-bucket.bucket
    }
  }
}
```

```
}
}
}
```

6.2 Distribución con CloudFront

Amazon CloudFront actúa como CDN y punto de terminación HTTPS para el *frontend*. La distribución se configura con las siguientes características:

- Origen: el bucket S3 con hosting web estático (*endpoint* HTTP del bucket S3 Website).
- Comportamiento de caché: TTL por defecto de 3600 segundos (1 hora), con política PriceClass_100 que limita la distribución a edge locations de Europa y Norteamérica, reduciendo el coste.
- Respuestas de error: los códigos 403 y 404 se redirigen a /index.html con código de respuesta 200, permitiendo que el enrutador de Angular gestione las rutas en cliente.
- Protocolo: redirect-to-https garantiza que todas las conexiones usan HTTPS.

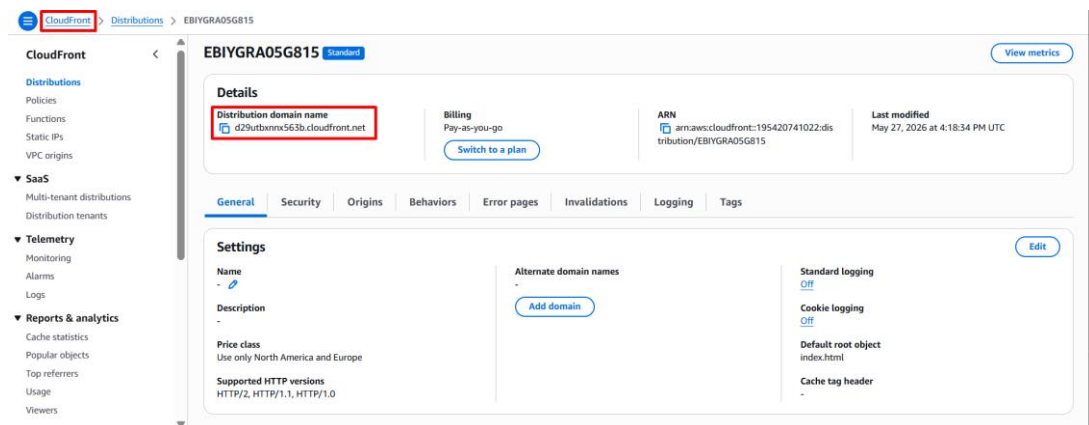


Figura 9. Captura de la consola de AWS mostrando la distribución CloudFront con su dominio y estado Deployed

La distribución incluye por defecto un dominio generado automáticamente por AWS con certificado SSL/TLS incluido, por lo que no es necesario disponer de un certificado propio para servir la aplicación con HTTPS. Si se desea utilizar un dominio personalizado, AWS Certificate Manager (ACM) permite solicitar y gestionar certificados de forma gratuita, que pueden asociarse a la distribución CloudFront para exponer la aplicación bajo una URL personalizada como <https://calendariofusion.com>. Cabe aclarar que lo gratuito es el certificado; el dominio deberá adquirirse a través de un registrador externo como GoDaddy o Namecheap, o registrarse directamente con AWS Route 53.

6.3 Proceso de despliegue

El despliegue completo del sistema se realiza en dos pasos:

Paso 1 — Compilación del *backend*:

```
cd aws-lambda-01
mvn clean package
```

Este comando compila el código Java, ejecuta los tests unitarios y empaqueta el JAR con todas las dependencias mediante el plugin Maven Shade.

Paso 2 — Inicialización de Terraform:

```
terraform init
```

Este comando inicializa el directorio de trabajo de Terraform, descarga los plugins de los proveedores necesarios (AWS) y configura el *backend* remoto de estado en S3. Solo es necesario ejecutarlo la primera vez o cuando se añaden nuevos proveedores

Paso 3 — Revisión del plan de cambios:

```
terraform plan
```

Este comando calcula y muestra el conjunto de cambios que Terraform aplicará sobre la infraestructura existente, sin ejecutar ninguna modificación real. Es fundamental revisar el plan antes de aplicar para confirmar que los recursos que se van a crear, modificar o destruir son los esperados.

Paso 4 — Aplicación de los cambios:

```
terraform apply
```

Terraform muestra de nuevo el plan de cambios y solicita confirmación explícita escribiendo yes. Tras la confirmación, aplica todos los cambios de infraestructura, sube el JAR compilado a S3, compila el *frontend* Angular y sincroniza los ficheros estáticos con el bucket S3. Los outputs al final del apply muestran las URLs del sistema:

```
api_url      = "https://8985gpp6l3.execute-api.us-east-1.amazonaws.com/"
cloudfront_url = "https://d29utbxnnx563b.cloudfront.net"
frontend_url  = "aws-tfg-frontend-plfz.s3-website-us-east-1.amazonaws.com"
```

24

35

7 Análisis de Costes AWS

En este capítulo se analiza el coste de la infraestructura AWS de CalendarFusion en función del número de usuarios activos y calendarios gestionados. Se presentan tres escenarios representativos: uso mínimo (desarrollo/pruebas), uso medio (100 usuarios activos) y uso a escala (1.000 usuarios activos).

7.1 Supuestos del modelo de costes

El modelo se basa en los siguientes supuestos:

- Un usuario activo genera de media 10 peticiones al mes a la API (creación/eliminación de fusiones, reconexiones de cuentas).
- Cada usuario tiene de media 2 fusiones activas, cada una con 2 fuentes de calendario.
- El refresco periódico de EventBridge se ejecuta cada 15 minutos: 4 invocaciones/hora $\times$ 24 h $\times$ 30 días = 2.880 invocaciones/mes.
- Cada invocación de refresco procesa todas las fusiones activas: 2 $\times$ N_usuarios fusiones por invocación.
- Duración media de cada invocación Lambda: 2 segundos con 512 MB de memoria.
- Tamaño medio de cada fichero ICS generado: 50 KB.

El almacenamiento DynamoDB se considera despreciable (<1 MB para < 1.000 usuarios). El coste por operaciones de lectura y escritura (RCU/WCU) también resulta despreciable en los tres escenarios analizados: con 10 peticiones por usuario al mes y 2.880 invocaciones de refresco, el volumen total de operaciones se mantiene dentro del nivel gratuito permanente de AWS en modo PAY PER REQUEST (1 millón de solicitudes gratuitas al mes).

7.2 Costes por servicio

A continuación se detallan los costes estimados para cada servicio AWS en los tres escenarios. Todos los precios corresponden a la región us-east-1 (N. Virginia) en mayo de 2026.

Servicio	Desarrollo	100 usuarios	1.000 usuarios	Notas
Lambda	~0 €	~0.02 €	~0.18 €	1M invoc. gratis/mes
API Gateway	~0 €	~0.01 €	~0.10 €	1M llamadas gratis/mes
DynamoDB	~0 €	~0.01 €	~0.05 €	25 GB gratis/mes

S3 (almacenamiento)	~0 €	~0.01 €	~0.10 €	5 GB gratis 12 meses
S3 (transferencia)	~0 €	~0.02 €	~0.18 €	Peticiones GET ICS
CloudFront	~0 €	~0.01 €	~0.08 €	1 TB gratis 12 meses
Secrets Manager	~0.40 €	~0.40 €	~0.40 €	0.40 €/secreto/mes
EventBridge	~0 €	~0 €	~0 €	Siempre gratuito
TOTAL ESTIMADO	~0.40 €	~0.48 €	~1.09 €	

Tabla 2: Estimación de costes mensuales en AWS por escenario de uso.

El secreto almacenado en Secrets Manager corresponde únicamente a las credenciales OAuth2 globales de la aplicación con Google Calendar API (client_id y client_secret), no a las claves individuales de cada usuario. Los tokens de acceso por usuario se almacenan en DynamoDB.

7.3 Análisis y conclusiones del modelo de costes

El análisis de costes revela tres conclusiones relevantes. En primer lugar, el coste fijo dominante en todos los escenarios es el secreto de Secrets Manager (0,30 €/mes), que no varía con el número de usuarios. En segundo lugar, los costes variables son extraordinariamente bajos gracias al modelo de facturación por demanda de Lambda y DynamoDB y a los generosos niveles gratuitos de AWS: para 1.000 usuarios activos el coste total estimado es inferior a 1,10 €/mes. En tercer lugar, el componente de coste que crecerá más rápido con la escala es Lambda (invocaciones de refresco), que puede optimizarse aumentando el intervalo de refresco o implementando un sistema de refresco por demanda.

Los datos reales de facturación de AWS durante el periodo de desarrollo del proyecto validan estas estimaciones. La figura siguiente muestra el desglose de costes del mes de mayo de 2026, correspondiente al periodo de mayor actividad del proyecto, con un coste total de 0,37 \$ frente a los 0,42 \$ previstos por el modelo:

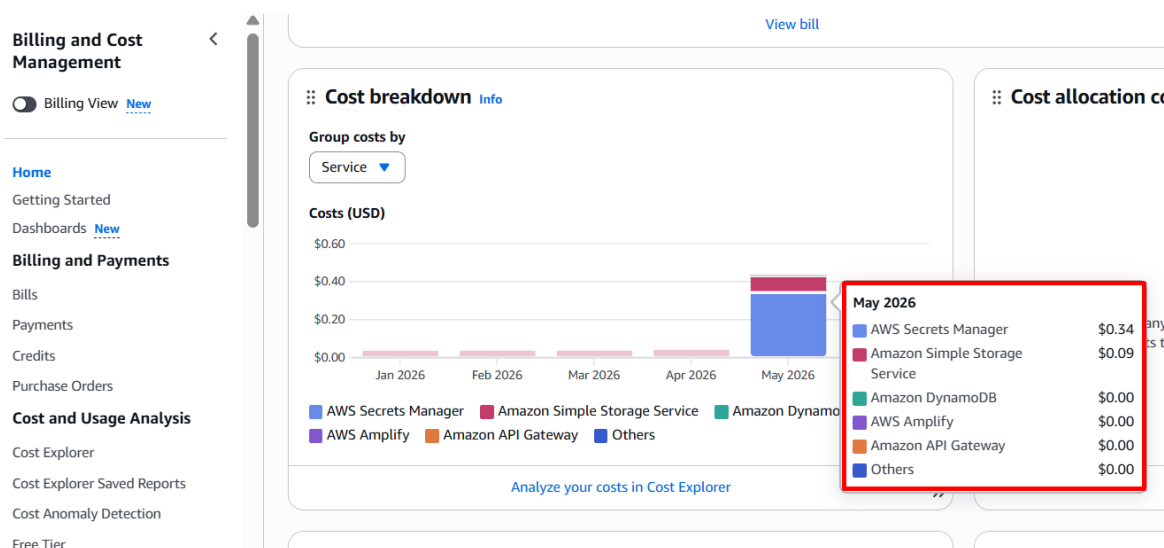


Figura 10. Desglose de costes reales en AWS – Mayo 2026. AWS Secrets Manager representa el mayor coste fijo (0,30\$), Amazon S3 el mayor coste variable (0,07\$) mientras que DynamoDB, API Gateway y Lambda registran coste 0 al mantenerse dentro del nivel gratuito de AWS.

En comparación con una arquitectura equivalente basada en servidores dedicados o contenedores, la arquitectura *serverless* presenta ventajas económicas significativas, especialmente a escala reducida. Una instancia EC2 de gama baja (t3.small) tiene un coste fijo de aproximadamente 15-20 €/mes, y una instancia de gama media (t3.medium) ronda los 30-40 €/mes, independientemente del tráfico que reciba. Para los tres escenarios analizados, el coste *serverless* oscila entre 1 y 50 €/mes, lo que representa un ahorro de entre el 30% y el 90% respecto a una instancia dedicada equivalente.

Además, una instancia dedicada tiene una capacidad máxima de usuarios concurrentes limitada por sus recursos, por lo que a medida que crece el número de usuarios sería necesario escalar verticalmente (instancias más potentes) u horizontalmente (más instancias), con el consiguiente incremento de coste no lineal y mayor complejidad operativa. Determinar cuántos usuarios soporta una única instancia requiere además un análisis de carga previo, y sobredimensionar la infraestructura para absorber picos de tráfico implica pagar por recursos ociosos la mayor parte del tiempo.

La arquitectura *serverless*, en cambio, escala de forma automática y lineal con el número de usuarios y peticiones, sin necesidad de planificación de capacidad ni intervención manual. AWS Lambda responde a cada invocación de forma independiente, por lo que el coste crece proporcionalmente al uso real y nunca se paga por capacidad no utilizada. Esto convierte a la arquitectura *serverless* en la opción más eficiente económicamente para aplicaciones con tráfico variable o en fases tempranas de crecimiento, donde el número de usuarios aún no justifica el coste fijo de una infraestructura dedicada.

8 Resultados y Conclusiones

8.1 Resultados obtenidos

Al término del trabajo se ha desarrollado e implantado en producción una herramienta web funcional, CalendarFusion, que cumple todos los objetivos planteados en el plan de trabajo. A continuación se describe el estado final de cada objetivo específico.

Integración con Google Calendar API y CalDAV. La herramienta soporta la conexión de cuentas de Google Calendar mediante el flujo OAuth 2.0 completo, y de cualquier servidor CalDAV compatible (incluyendo Apple iCloud) mediante credenciales Basic Auth sobre HTTPS. La integración con iCloud requirió la implementación de un mecanismo de desescapado de entidades XML específico de la respuesta del servidor de Apple, un caso no documentado en la especificación oficial de CalDAV.

Motor de fusión con filtros y transformaciones. El motor de fusión permite combinar eventos de cualquier número de fuentes, aplicar prefijos y sufijos configurables al título de cada evento por fuente, y excluir eventos por palabras clave en el título o descripción. La salida es un fichero ICS completamente conforme al estándar RFC 5545, verificado mediante suscripción exitosa desde Google Calendar, Microsoft Outlook y Apple Calendar.

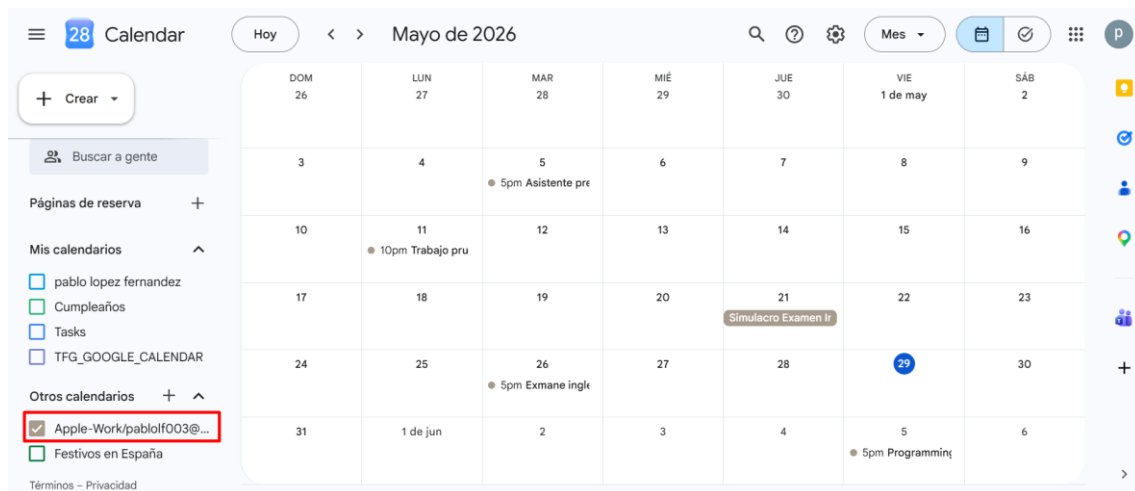


Figura 11. Captura del ICS generado en el navegador mostrando eventos de Google Calendar y Apple iCloud fusionados con prefijos distintos

Prevención de circularidades. El sistema detecta y descarta eventos procedentes de ICS generados por la propia herramienta mediante la comprobación del campo PRODID, evitando la duplicación indefinida de eventos en escenarios de fusiones anidadas.

Arquitectura serverless en AWS. El sistema está desplegado en producción sobre AWS con la arquitectura descrita: Lambda, API Gateway, DynamoDB, S3

y EventBridge. El refresco automático cada 15 minutos mediante EventBridge garantiza que los calendarios suscritos se actualicen periódicamente sin intervención del usuario.

SPA Angular con CloudFront. La interfaz de usuario está disponible públicamente en <https://d29utbxnnx563b.cloudfront.net> con HTTPS. La experiencia de usuario permite conectar cuentas, crear y gestionar fusiones, y obtener la URL ICS en un flujo de tres pasos.

Infraestructura como Código con Terraform. Toda la infraestructura AWS se define declarativamente en Terraform. El despliegue completo del sistema, desde cero hasta producción, se realiza en dos comandos (`mvn package` y `terraform apply`).

Análisis de costes. El modelo de costes demuestra que la infraestructura es viable económicamente para hasta 1.000 usuarios activos por menos de 1,10 €/mes en costes variables, con el coste fijo dominado por Secrets Manager (0,40 €/mes).

8.2 Líneas de trabajo futuro y mejoras

A partir del trabajo realizado se identifican las siguientes líneas de mejora y ampliación:

- **Integración con Microsoft Graph para Outlook:** La arquitectura modular del *backend* facilita la incorporación de nuevos proveedores. Como siguiente paso natural, la integración con Microsoft Graph permitiría dar soporte a calendarios de Outlook y Microsoft 365, ampliando significativamente la base de usuarios potenciales.
- **Tubería de procesos (Pipeline) CI/CD con GitHub Actions:** Actualmente el despliegue se realiza ejecutando Terraform de forma manual. La implementación de un *pipeline* CI/CD con GitHub Actions permitiría automatizar el proceso de compilación (*build*), test y despliegue en cada envío de código (*push*) a la rama principal, reduciendo el riesgo de errores manuales y agilizando el ciclo de desarrollo.
- **Webhooks de Google Calendar:** El refresco de eventos se realiza actualmente de forma periódica. Google Calendar API soporta webhooks que notifican cambios en tiempo real, lo que permitiría actualizar el calendario fusionado de forma inmediata ante cualquier modificación, eliminando la latencia del refresco periódico.
- **Visualización del calendario en la SPA:** Actualmente el calendario fusionado solo es accesible mediante suscripción a la URL ICS desde una plataforma externa. Incorporar una vista de calendario directamente en la SPA, mediante una librería como FullCalendar, mejoraría

significativamente la experiencia de usuario al no requerir el uso de una aplicación de calendario externa para visualizar el resultado.

- **Cobertura de tests:** La integración de JaCoCo como herramienta de análisis de cobertura permitiría identificar qué partes del código no están cubiertas por los tests actuales y establecer umbrales mínimos de cobertura como requisito en el *pipeline* CI/CD.
- **Control de circularidad por usuario:** La detección de circularidad actual opera a nivel de firma del fichero ICS, lo que impide también el uso cruzado entre usuarios. Un control más fino a nivel de usuario permitiría distinguir entre calendarios propios y ajenos, abriendo la posibilidad de colaboración controlada entre usuarios de la plataforma.

8.3 Conclusiones

El desarrollo de este TFG ha permitido demostrar que es posible construir una solución completa de integración de calendarios multi-proveedor sobre una arquitectura *serverless* en AWS, con despliegue automatizado mediante Terraform, que es funcional, escalable y económicamente viable.

Desde el punto de vista técnico, el trabajo ha permitido profundizar en los estándares abiertos de calendario (iCalendar y CalDAV) y en sus particularidades de implementación en los distintos proveedores, especialmente las divergencias entre la especificación RFC 4791 y el comportamiento real de Apple iCloud CalDAV. La integración de OAuth 2.0 con Google Calendar API ha puesto de manifiesto la complejidad de los flujos de autorización delegada en aplicaciones web.

Desde el punto de vista de la ingeniería de software, el proyecto ha aplicado principios de diseño modular que facilitan la extensión futura del sistema: la incorporación de un nuevo proveedor de calendarios (por ejemplo, Microsoft Graph para Outlook) requeriría únicamente implementar un nuevo método `fetchOutlook()` en `CalendarMergeService` y añadir el caso correspondiente en el método `merge()`, sin modificar el resto del sistema.

9 Análisis de Impacto

En este capítulo se analiza el impacto potencial de los resultados obtenidos en este TFG en diferentes contextos, destacando los beneficios esperados y los posibles efectos perjudiciales, con referencia a los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030.

9.1 Impacto personal

Para el autor, el desarrollo de este TFG ha supuesto una inmersión práctica en las tecnologías de computación en la nube, arquitectura *serverless* e infraestructura como código, competencias altamente demandadas en el mercado laboral actual. El proyecto ha permitido aplicar de forma integrada conocimientos adquiridos a lo largo del grado en áreas como diseño de APIs REST, persistencia NoSQL, desarrollo *frontend* con frameworks modernos y seguridad en aplicaciones web (OAuth 2.0, HTTPS, gestión de secretos).

A nivel personal, el proyecto resuelve también una necesidad real del propio autor: la gestión unificada de calendarios de múltiples cuentas y servicios en un entorno de escritorio web.

9.2 Impacto empresarial

La herramienta desarrollada tiene potencial de aplicación en entornos profesionales donde los empleados trabajan con múltiples cuentas de calendario (personal y corporativa, de distintos proveedores). La posibilidad de configurar calendarios fusionados con filtros por categoría permite, por ejemplo, crear vistas unificadas que excluyan eventos personales de la vista profesional, o agregar calendarios de distintos departamentos en una vista ejecutiva.

La arquitectura *serverless* garantiza que el coste de la infraestructura escala linealmente con el uso, lo que la hace especialmente adecuada para startups y pymes que no quieren asumir costes fijos de infraestructura antes de validar el producto.

9.3 Impacto social

La gestión eficiente del tiempo es un factor determinante en el bienestar personal y en la productividad. Herramientas que simplifican la organización de la agenda personal y profesional contribuyen a reducir la carga cognitiva asociada a la gestión de múltiples calendarios. En particular, para personas que combinan actividades laborales, académicas y personales en distintas plataformas, la posibilidad de tener una vista unificada y actualizada de sus compromisos puede mejorar significativamente su gestión del tiempo.

9.4 Impacto económico

26 El modelo de costes analizado en el capítulo anterior demuestra que la solución es viable económicamente incluso para un servicio de bajo tráfico. La arquitectura *serverless* elimina el coste de infraestructura en reposo, lo que reduce significativamente la barrera de entrada para ofrecer el servicio comercialmente. Un servicio SaaS basado en esta arquitectura podría ser rentable con un número relativamente pequeño de usuarios de pago.

9.5 Impacto medioambiental

La arquitectura *serverless* tiene un impacto medioambiental potencialmente inferior a las arquitecturas basadas en servidores dedicados, ya que los recursos de computación solo se consumen durante el tiempo de procesamiento efectivo, sin mantener instancias activas en reposo. Los grandes proveedores de nube como AWS han adquirido compromisos de alimentar sus centros de datos con energías renovables, lo que mejora el perfil medioambiental de los servicios desplegados en su infraestructura.

14 Este trabajo se alinea con el **ODS 12 (Producción y Consumo Responsables)** al promover el uso eficiente de los recursos computacionales mediante una arquitectura que minimiza el consumo de recursos en reposo.

9.6 Impacto cultural

La adopción de estándares abiertos (iCalendar, CalDAV, RFC 5545, RFC 4791) como base de la solución promueve la interoperabilidad y la independencia de proveedor, valores fundamentales en la cultura del software libre y los estándares abiertos. La herramienta permite a los usuarios gestionar sus datos de calendario de forma independiente de las plataformas propietarias de Google, Apple o Microsoft, contribuyendo a reducir el efecto de lock-in de los grandes proveedores.

1 Este trabajo se alinea con el **ODS 9 (Industria, Innovación e Infraestructura)** al desarrollar una solución técnica innovadora que aprovecha infraestructura cloud moderna para resolver un problema de interoperabilidad real, y con el **ODS 17 (Alianzas para Lograr los Objetivos)** al basarse en estándares abiertos que fomentan la colaboración y la interoperabilidad entre diferentes ecosistemas tecnológicos.

10 Bibliografía

- [1] B. Desruisseaux, Ed., "Internet Calendaring and Scheduling Core Object Specification (iCalendar)," RFC 5545, Internet Engineering Task Force, Sep. 2009. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5545>
- [2] C. Daboo, B. Desruisseaux, L. Dusseault, "Calendaring Extensions to WebDAV (CalDAV)," RFC 4791, Internet Engineering Task Force, Mar. 2007. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4791>
- [3] Google LLC, "Google Calendar API v3 Reference," Google Developers, 2024. [Online]. Available: <https://developers.google.com/calendar/api/v3/reference>
- [4] D. Hardt, Ed., "The OAuth 2.0 Authorization marco de trabajo (*Framework*)," RFC 6749, Internet Engineering Task Force, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [5] Microsoft Corporation, "Microsoft Graph REST API v1.0," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/graph/api/resources/calendar>
- [6] Flexibits Inc., "Fantastical — Calendar and Tasks," 2024. [Online]. Available: <https://flexibits.com/fantastical>
- [7] Readdle Inc., "Calendars 5 — Calendar and Planner," 2024. [Online]. Available: <https://readdle.com/calendars>
- [8] Merge Calendar, "Merge Calendar — ICS Calendar Merger," 2024. [Online]. Available: <https://www.mergecalendar.com>
- [9] Zapier Inc., "Zapier Automation Platform," 2024. [Online]. Available: <https://zapier.com>
- [10] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [11] Amazon Web Services, "Amazon DynamoDB Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/>
- [12] HashiCorp Inc., "Terraform Documentation," 2024. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
- [13] Google LLC, "Spring Boot on AWS Lambda," aws-serverless-java-container GitHub, 2024. [Online]. Available: <https://github.com/aws-labs/aws-serverless-java-container>
- [14] Angular Team, "Angular Documentation," 2024. [Online]. Available: <https://angular.dev>

6

[15] Amazon Web Services, "Amazon CloudFront Developer Guide," 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/>